

IoT Lab 1 — MQTT Ingestion and Visualization

September 26, 2026

Abstract

This short report summarizes the work performed to ingest DHT sensor data via MQTT, process it with Telegraf, store it in InfluxDB, and visualize it with Grafana. The document includes key decisions, problems encountered and solved, results, and concise conclusions. Image placeholders are included for figures generated during the project.

Work Done

The following points describe the main tasks, decisions, rationale and noteworthy details from the development process.

- **Arduino / Sensor integration:**

- Implemented DHT11 reading using the `DHT.h` library. Readings validate with `isnan()` and the code retries or skips invalid values.
- Decision: publish a single JSON payload per device to a single topic (e.g. `sensor/esp01`). Reason: a single JSON topic simplifies downstream parsing and reduces the number of measurements and queries.

- **MQTT broker (Mosquitto) in Docker:**

- Ran Mosquitto inside Docker for reproducibility, isolation, and easy updates.
- Persisted configuration and data using bind mounts to the host file system.

- **Telegraf configuration and data treatment:**

- Configured the MQTT consumer to subscribe to `sensor/#` and set `data_format = "json"` so JSON keys map to fields and tags.
- Promoted `device` and `location` to `tag_keys` for efficient grouping and filtering in InfluxDB/Grafana.
- Set `name_override = "mqtt_consumer"` to keep a consistent measurement name.
- Problem found: initial config used `data_format = "value"` which failed for JSON payloads; solution was to switch to `"json"`.

- **InfluxDB and Grafana provisioning:**

- Used InfluxDB 1.8 with InfluxQL for compatibility with existing dashboards.
- Ensured Grafana provisioning files contain `secureJsonData.password` and correct datasource versioning.
- Problem found: Grafana query parsing errors — debugged using the Influx CLI and the Grafana provisioning API, then corrected quoting and query macros.

- **Dashboards and visualizations:**

- Created a time-series dashboard with separate panels for temperature and humidity.
- Planned a Geomap panel using numeric `lat` and `lon` fields and the latest temperature as the value for styling.

Results

Summary of results and the assets produced.

- **Data ingestion:** Telegraf successfully writes to InfluxDB (observed HTTP 204 on POST). Influx CLI queries return expected values, for example the last temperature and coordinates per device.
- **Visualizations created:**
 - **Temperature timeseries:** a Grafana timeseries panel plotting mean values aggregated by the dashboard interval. Placeholder image below.



Figure 1: Temperature timeseries (replace with generated plot).

- **Humidity timeseries:** similar to temperature, with percentage unit and thresholds. Placeholder image below.

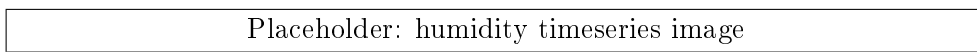


Figure 2: Humidity timeseries (replace with generated plot).

- **Geomap (planned):** a map panel that expects a table with columns `lat`, `lon`, and a value (e.g. `last(temperature)`). Insert a screenshot after provisioning.

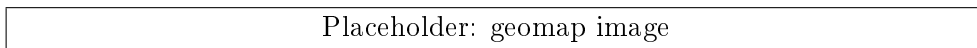


Figure 3: Geomap placeholder. Query should return `lat`, `lon`, and the value per device.

- **Example JSON payload used for tests:**

```
{"device":"esp01","location":"Room1","lat":41.3879,"lon":2.16992,"temperature":24.5,"humidity":50.3}
```

- **Example test commands (run on host):**

```
mosquitto_pub -h localhost -t sensor/esp01 -m '{"device":"esp01","location":"Room1","lat":41.3879,"lon":2.16992,"temperature":24.5,"humidity":50.3}'
docker compose exec influxdb influx -username admin -password adminpass -database mqttddb -execute "SELECT last(\"temperature\"), last(\"lat\"), last(\"lon\") FROM \"mqtt_consumer\" WHERE \"device\"='esp01'"
```

Answers to the Questions

Concise, direct answers to the original project questions.

- **Why is MQTT suitable for IoT?**
 - MQTT is lightweight and has a small header, reducing bandwidth and power use — ideal for battery-powered sensors.
 - The pub/sub model decouples producers and consumers, reducing polling and unnecessary traffic.
- **How does pub/sub improve efficiency?**
 - Publishers send updates only when values change; subscribers receive updates only when published, avoiding repeated requests.
- **Advantages of running Mosquitto in Docker:**
 - Simple installation, isolation from host, reproducible deployment, persistent volumes for config/data, and easy updates.

Conclusions

The chosen design (JSON single-topic payloads, Telegraf JSON ingestion, device/location as tags) simplifies ingestion and dashboarding, enables a Geomap visualization using numeric coordinates, and reduces parsing complexity. Recommended next steps:

- Provision a Grafana Geomap dashboard JSON using a table query that returns `lat`, `lon`, and the latest `temperature` per device.
- Harden MQTT with authentication and TLS before exposing the broker to untrusted networks.
- Store device metadata in a small static measurement or external config for large fleets.

Generated report. Replace images in the `figures/` directory with the actual exported dashboard screenshots or plots.